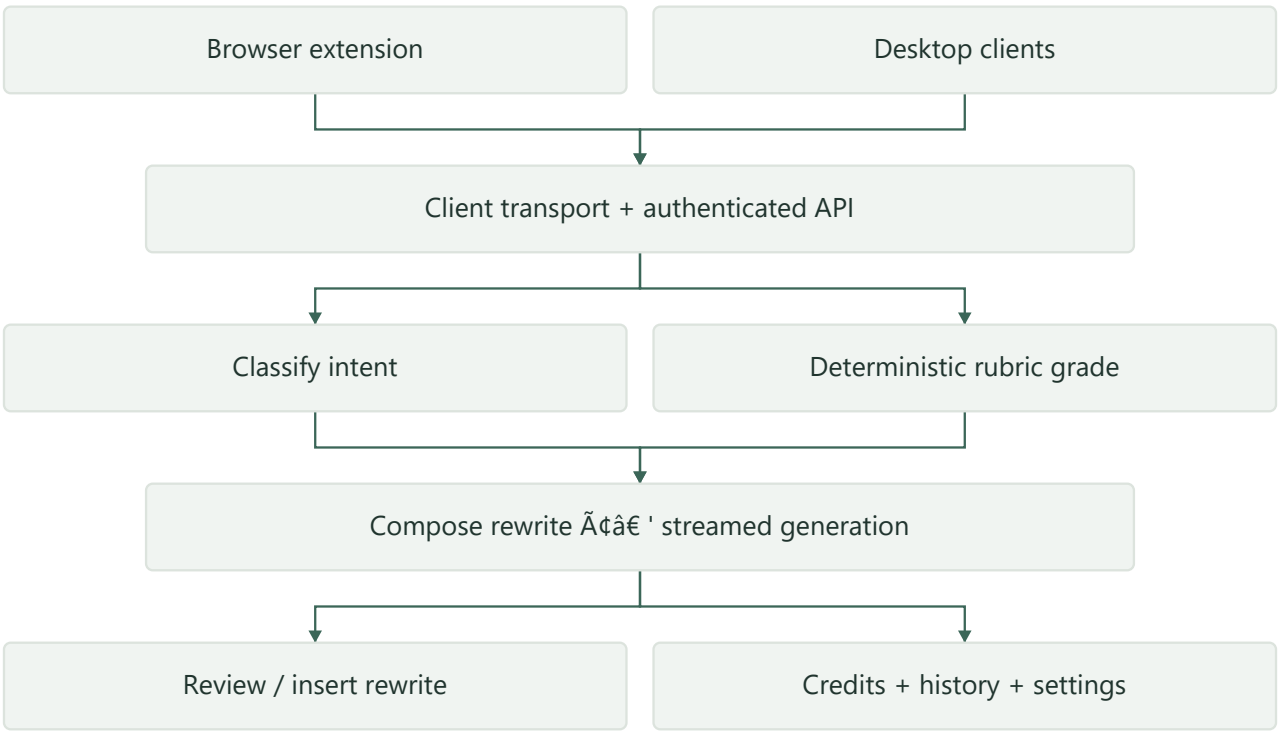
Amplify AI

Prompt refinement across AI interfaces

Browser and desktop client code for prompt classification, rubric-based grading, and streamed rewriting through a shared backend.

Architecture



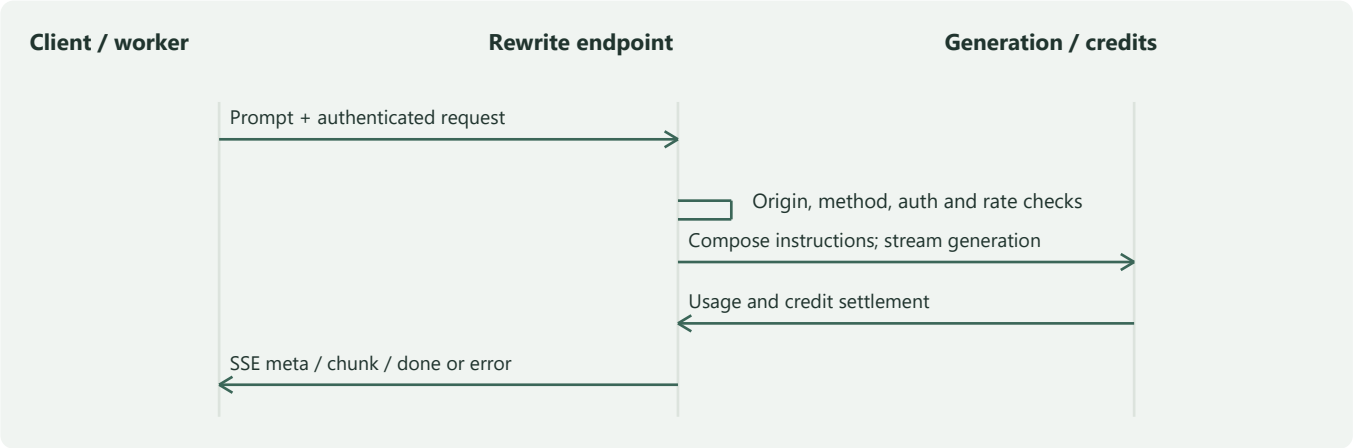
Component overview derived from the supplied source; version-specific details may differ.

Component	Implementation	Role
Client layer	Extension + desktop source	Capture, review, apply
Transport	Background messaging / API client	Request and stream boundaries
Evaluation	Rule-based rubric grader	Issue list and product score
Backend	Auth, prompt composition, SSE	Streamed rewrite generation
Persistence	Supabase schema and handlers	Settings, credits, history

Product implementation

Request lifecycle & streaming

Amplify AI spans browser/desktop clients and a backend. The client captures a prompt and presents feedback, while the backend controls authenticated rewriting, settings and credit accounting. The source keeps the browser content script behind a background-worker transport boundary rather than placing every network action in the page context.



Transport and response semantics

The extension API client defines typed contracts for classification, grading, settings, credit state and rewriting. The rewrite route sends server-sent events: meta carries request information, chunk carries text deltas, done marks completion, and error communicates failure. The client must distinguish partial text from a completed rewrite and preserve request identity through the stream.

Accounting boundary

The server checks credit availability before generation and settles usage before sending its completion event in the inspected route. This ordering lets the client refresh credit state after completion. It is not by itself proof of concurrency-safe accounting, disconnect recovery or exactly-once billing; those behaviors depend on the credit store and integration tests.

Boundary	Responsibility
Page to worker	Capture/insert UI state and route authorized requests
Worker to API	Authenticated transport and typed payloads
API to generation	Instruction composition, streaming, usage and errors

Source / interpretation: Amplify-AI-Build/apps/extension/lib/api-client.ts; ToneShift/api/rewrite.js; backend credit helpers. Sequence condenses the inspected route, not a claim about every deployed client.

Rubric grading as an algorithm

The grading layer is deterministic for a fixed prompt, category and rubric version. Its value is an inspectable set of failed checks rather than an unexplained model judgment. The score is useful product feedback, but its interpretation depends on what the rubric actually rewards.

Illustrative rubric penalties only: failed weights 20 + 15 + 0 produce score 65. These are not measured user results or asserted production weights.

Score construction

gradeWithRubric iterates through rubric checks, calls runCheck, adds each failed weight to a penalty and emits an issue containing the check ID, weight and optional description. Score = max(0, 100 - sum of failed weights). The passed flag compares this score with the category threshold. Category and rubric_version travel with the result, which makes changes in evaluation rules visible.

Why zero-weight issues matter

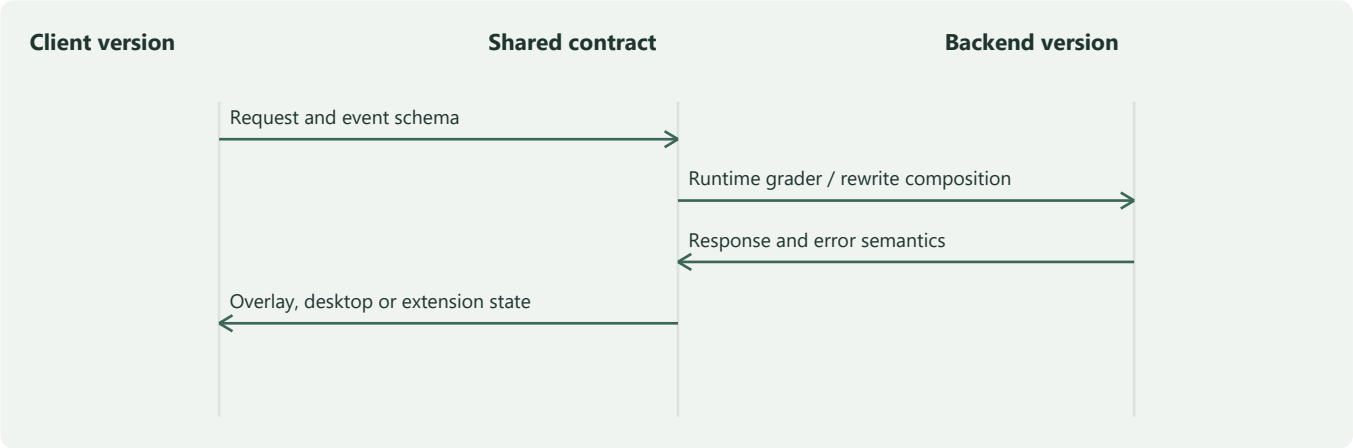
A failed check with weight zero appears in the issue list but does not lower the score. That lets the interface give advisory feedback independently of the numeric rating. Conversely, a higher post-rewrite score only shows that the rewrite satisfies more of this rubric; it does not prove a better answer from a downstream model, lower cost or improved user outcomes.

Evaluation layer	Suitable interpretation
Rubric result	Compliance with versioned product checks
Before/after grading	Movement under the same rubric and category
User task outcome	Requires independent task-level evaluation, not score alone

Source / interpretation: Amplify-AI-Build/packages/grader/src/grader.ts; associated rubric loader/check dispatch. Numeric example constructed to explain the formula.

Multi-client system & verification

The local workspace contains Amplify-AI-Build client/shared packages and a ToneShift backend with account, billing, settings and history surfaces. Treating them as explicit components is more accurate than presenting one repository as a complete deployed product.



Build-time and runtime dependencies

ToneShift retains generated JavaScript and declaration files for shared logic. Some server imports depend on those runtime outputs, so they cannot be removed merely because TypeScript sources also exist. A release should document what is generated, what command generates it, and which deployment artifact consumes it. Native desktop build outputs are a separate concern from reusable source.

Tests that would establish the product flow

A bounded integration suite should verify invalid-origin/auth rejection, exhausted-credit behavior, stream completion/error handling, interrupted generation, and client refresh after settlement. A separate compatibility matrix should identify which extension/desktop build was tested against which backend revision. These are proposed verification targets; the portfolio pass inspected source and syntax rather than running live account or billing actions.

Material	Release interpretation
Extension and desktop clients	Distinct capture, display and installation surfaces
Shared grader and resources	Versioned logic used by clients or server
Backend and database schema	Persistence and service behavior; deployment evidence separate

Source / interpretation: Amplify-AI-Build workspace; ToneShift API, shared runtime and schema. Contribution ownership and deployment/adoption evidence remain attached to their original repositories.

Evidence & source

Implementation, supporting records, and the boundary of the claim

Area	Evidence	Reference
Local components	ToneShift + Amplify-AI-Build	Separate backend/client boundaries
Evaluation mechanism	100 minus weighted failed checks	Deterministic grader source
Outcome status	Adoption and release claims not added	Release/deployment evidence to be supplied

Selected implementation files

File / record	Path / reference
grader.ts	Amplify-AI-Build/packages/grader/src/grader.ts
api-client.ts	Amplify-AI-Build/apps/extension/lib/api-client.ts
rewrite.js	ToneShift/api/rewrite.js

Selected client contracts, grading, and rewrite route. Shared/client code retains its original repository paths; private configuration and campaign lists are excluded.

Evidence boundary

The rubric score is a product-specific heuristic, not a universal measure of prompt quality. Backend, client versions, and your contribution across the repositories must remain explicit.

Technical note

The extension routes requests through a background worker. The rewrite endpoint performs origin/auth/rate checks before sending SSE metadata, chunks, completion, or errors. Billing state is handled server-side.